DB.PY LINE-BY-LINE EXPLANATION

db.py is the data access layer. It can use MySQL when configured, but falls back to built-in sample listings and in-memory demo inquiries when MySQL is unavailable.

IMPORTS AND OPTIONAL MYSQL

Lines 1-3: import environment access, a context helper for closing connections, and pathlib for .env loading.

Lines 5-10: try to import mysql.connector. If it fails, set mysql to None and make MySQLError a generic Exception fallback.

DEMO DATA

Lines 13-98: SAMPLE_LISTINGS defines four built-in rental listings used when MySQL is not available.

Lines 14-34: first listing, a featured 2BHK apartment in Bengaluru.

Lines 35-55: second listing, a furnished studio in Pune.

Lines 56-76: third listing, a featured villa in Hyderabad.

Lines 77-98: fourth listing, a shared room in Delhi.

Line 100: DEMO_INQUIRIES stores submitted inquiries in memory during demo mode.

SHARED CONSTANTS

Lines 102-107: LISTING_COLUMNS holds the column list used by MySQL SELECT queries.

Line 109: BOOLEAN_FIELDS lists fields that should be converted to Python bool values after database reads.

ENVIRONMENT AND CONNECTION

Lines 112-125: load_env reads key=value pairs from .env, skips blank/comment lines, and sets missing environment variables.

Lines 127-129: _mysql_configured checks whether any MySQL-related environment switch/user is present.

Lines 131-143: _db_config builds the host, port, user, password, and database config from MYSQL_* or DB_* variables.

Lines 146-152: _connect returns None when MySQL is unavailable/unconfigured or when the connection attempt fails.

Lines 155-160: storage_label returns Demo data or MySQL so the UI can show the active source.

NORMALIZATION HELPERS

Lines 163-170: _clean_int converts filter values to integers, returning None for missing or invalid input.

Lines 172-180: _normalize_listing copies a DB row to a dict and converts is_featured/is_available to booleans.

Lines 183-186: _sample_listing returns a copy of a sample listing so callers do not mutate the original data.

SEARCH LISTINGS

Lines 189-193: search_listings opens a connection and falls back to sample search when no connection exists.

Lines 194-199: builds the base MySQL query for available listings and initializes query parameters.

Lines 201-206: read and normalize filters for city, type, furnishing, price range, and bedrooms.

Lines 208-225: append WHERE clauses only for filters the user supplied, using parameterized values.

Line 227: order results with featured listings first, then lower monthly rent first.

Lines 229-238: execute the query, normalize rows, fallback to sample data on MySQL errors, and close the cursor.

SINGLE LISTING AND INQUIRY

Lines 240-245: get_listing falls back to matching SAMPLE_LISTINGS by id when no MySQL connection exists.

Lines 247-265: query MySQL for one available listing, normalize it, fallback on errors, and close the cursor.

Lines 267-271: create_inquiry stores inquiry data in DEMO_INQUIRIES and returns False when MySQL is unavailable.

Lines 273-295: insert inquiry data into MySQL, commit on success, rollback on failure, and close the cursor.

SAMPLE SEARCH

Lines 298-305: _search_sample_listings starts from available sample listings and normalizes filter values.

Lines 307-324: apply the same filters in Python that the MySQL query applies in SQL.

Lines 326-331: return copied listings sorted with featured listings first and lower rent first.